Innopolis University

Confidence-Aware Adaptive Verification of LLM Recommendations in Introductory Programming Tasks

Student: Lidia Davydova

Supervisor: Paolo Ciancarini

November 1, 2026

Contents

Abstract **3**

1 Introduction **5**

1.1 Motivation 5

1.2 Research Problem 6

1.3 Research Gap 7

1.4 Conceptual Framework: An Explicit Escalation Rule 8

1.5 Research Questions 10

1.6 Approach 11

1.7 Contributions and Scope 11

1.8 Structure of the Thesis 13

2 Literature Review **14**

2.1 Introduction and Review Scope 14

2.2 Trust, Reliance, and Calibration 15

2.3 Interventions That Support Reliance 18

2.3.1 Explanations and Explainable AI 18

2.3.2 Cognitive Forcing and Expectation Setting 20

2.3.3 Adaptive and Selective Interventions 21

2.3.4 Comparative Evaluation 22
2.4 Independent Verification and Its Cost 24
2.5 Confidence as a Signal for Adaptive Support 25
2.6 Coordination and Governance of LLM-Based Systems 27
2.7 LLMs in Programming Education 28
2.8 Synthesis and Research Gap 28

References **30**

Abstract

Large language models (LLMs) are increasingly used in programming education, but their recommendations can be plausible yet incorrect, so the quality of human–AI collaboration depends on *appropriate reliance*: accepting correct recommendations and rejecting incorrect ones. Verification support can help users evaluate recommendations, and introductory programming is well suited to it because correctness can be checked through independent, execution-based evidence such as tests, traces, and counterexamples. Providing such evidence on every trial, however, introduces unnecessary interaction cost. This thesis investigates whether a simple, interpretable adaptive policy—gating verification availability on the participant's initial confidence—can reduce verification effort relative to always-available verification while preserving decision quality. The human–AI interaction is specified through an explicit *escalation rule* that fixes who recommends, who supplies independent verification evidence, who decides, and what is recorded; the three experimental conditions are three escalation policies within that single specification. A within-subject experiment compares no-verification, static, and adaptive conditions on a benchmark of introductory Python tasks with frozen, pre-validated LLM recommendations balanced between correct and incorrect. The two primary outcomes are final-answer accuracy and verification effort; reliance measures are secondary. Reliance on the LLM is distinguished from

agreement with an answer the participant already held, and adaptive verification is tested for non-inferiority in decision quality against static verification, including the boundary case of confident but mistaken participants. Expected contributions include trial-level evidence on the effort–quality trade-off of confidence-gated verification, a validated reproducible benchmark with an instrumented prototype, and a complete analysis pipeline.

Chapter 1

Introduction

1.1 Motivation

Large language models (LLMs) are increasingly used in programming education and software engineering to generate, explain, and debug code and to propose answers to programming tasks [1]–[3]. They can lower the cost of producing a first attempt, suggest alternative implementations, and explain unfamiliar concepts. At the same time, such systems can produce plausible but incorrect outputs: fluent code that fails to compile, passes some tests but not others, or satisfies the letter of a specification while missing its intent.

Because of this, the quality of the joint human–AI outcome depends not only on the accuracy of the model but on how the human partner responds to it. This is captured by the notion of *appropriate reliance* [4]: relying on correct recommendations and refraining from relying on incorrect ones, judged relative to the answer the user already holds [5]. Its two failure modes are well documented. *Over-reliance* occurs when a user who was initially correct accepts an incorrect recommendation; *under-reliance* occurs when a user who was initially incorrect

rejects a correct one. Both can persist even when additional information about the model is made available [6]–[8].

Verification support is a natural remedy: rather than asking users to trust or distrust the model on the basis of fluency, it lets them check a recommendation against independent evidence. Programming tasks suit this approach particularly well, because correctness can often be checked independently through execution, tests, traces, or counterexamples—evidence grounded in the task itself rather than in another LLM-generated justification. This distinction matters, because explanations help only when they sufficiently reduce the effort of checking the recommendation, and simply providing more information does not automatically improve appropriate reliance [7]–[10]. Moreover, feature-based explanations reduce over-reliance mainly for decisions that users can already evaluate themselves and lose their benefit as difficulty rises [11]; introductory programming provides exactly such checkable grounding.

The remaining difficulty is cost. Verification on every trial introduces unnecessary interaction effort: if evidence is displayed when it is not needed, it slows users down and consumes attention that could be spent elsewhere. This motivates providing support selectively: escalating verification effort when the system's confidence in the current answer is low [12], or when a user-side signal such as trust indicates that support is needed [13], using only information that is available before the correctness of the recommendation is known.

1.2 Research Problem

The research problem is how to provide verification support selectively, using only information available before the correctness of the LLM recommendation

is known. This thesis investigates whether a simple adaptive policy, based primarily on the user's initial confidence, can provide independent verification support at lower effort than always-available verification, and how verification effort trades off against decision quality.

The focus is a simple, interpretable policy—not a new LLM, a new verification algorithm, or an optimisation method. The contribution is therefore not the threshold rule itself; it is the controlled empirical evaluation of whether a simple confidence gate can reduce verification exposure while preserving appropriate reliance compared with always-available and no-verification baselines.

The problem is important because verification cost is paid on every trial, while its benefit is concentrated on the trials where the user is uncertain or wrong. Introductory programming amplifies both sides: novices are likely to over-rely on fluent but incorrect code, yet they are also the users least able to afford repeated checking. A policy that spends verification effort only where it is needed would make independent evidence practical in ordinary learning settings, not only in controlled studies.

1.3 Research Gap

Prior work has examined confidence mainly as an outcome to be calibrated, as an AI-side uncertainty signal for routing a case to stronger processing [12], or as a user-side trust signal for adapting explanations [13]. It has also shown that explanations and cognitive forcing can reduce over-reliance [8]–[10]. What is not known is whether the participant's own initial confidence can serve as a per-trial trigger for independent, execution-based verification, and whether such a gate preserves decision quality while reducing verification effort. Studies that

offer support on every trial do not measure the cost of unnecessary verification, while studies that manipulate AI-side signals do not test a user-side gate. Direct evidence on this effort–quality trade-off is therefore missing, especially for novices in introductory programming, where confident errors are common and independent evidence is cheap to produce.

The gap matters because a confidence-gated policy would be a lightweight, model-agnostic way to reduce interruption cost without weakening decisions. If the policy is not non-inferior, the result identifies when confidence should not be used as a gate, which is equally important for safe deployment. In introductory programming the stakes are concrete: novices are prone to over-reliance on plausible but incorrect code, and uncorrected errors can entrench misconceptions. The question therefore has to be answered empirically before such a policy is recommended in practice.

1.4 Conceptual Framework: An Explicit Escalation Rule

Coordination, in the account of Ciancarini [14], concerns who decides, with what evidence, and under whose authority, so that correct and incorrect recommendations can be managed by the surrounding system rather than by conversation alone. This thesis adopts that element in an explicit *escalation rule*: a specification of who supplies evidence, who decides, and when the evidence reaches the decider. The rule does not add any implementation; it makes the responsibilities already present in the design explicit. It has five parts:

- **Recommender.** A fixed, offline LLM snapshot produces the candidate an-

swer for each task. The recommender has no access to the experimental condition, the verification policy, or the participant.

- **Evidence provider.** The researchers supply independent, execution-based verification artefacts (tests, traces, counterexamples) derived from the task specification and reference answer, not from the recommender. The evidence provider never states whether the recommendation is correct.

- **Decider.** The participant holds decision authority: they record an initial answer, a confidence rating, and a final judgment.

- **Escalation rule.** The rule specifies when the evidence provider's output is escalated to the decider. Decision authority is never reassigned: the participant decides in every condition, and only the flow of evidence is conditional. Under the adaptive policy the escalation condition is $c < \tau$ on the participant's initial confidence; under the static policy evidence is always escalated; under no-verification it never is.

- **Audit and record.** The instrumented prototype records the task, condition, escalation decision, evidence availability and examination, timestamps, and the correctness label. The correctness label is kept only in the analysis dataset and is never shown to the decider.

The three experimental conditions are therefore *three escalation policies within the same specification*: only the escalation rule is changed, while the recommender, the evidence provider, the decider, and the audit record are held fixed. This keeps the implementation unchanged and makes the comparison a comparison of escalation policies rather than of different systems.

1.5 Research Questions

Primary RQ. Does confidence-aware adaptive verification reduce verification effort relative to always-available (static) verification in introductory programming tasks, while preserving decision quality relative to static verification (non-inferiority) and final-answer accuracy relative to no verification?

RQ1 (primary). How do no, static, and adaptive verification affect final-answer accuracy?

RQ2 (primary). How does the verification effort of adaptive verification (verification availability, evidence-examination rate, evidence-examination time) compare with static verification, and is decision quality preserved?

RQ3 (exploratory). How do the effort and effectiveness of the adaptive policy differ across initial confidence levels, task types, and the confident-but-mistaken stratum?

The primary hypotheses follow from the research questions.

- **H1 (quality).** Adaptive verification is non-inferior to static verification on final-answer accuracy within the pre-specified margin δ_Q.
- **H2 (effort).** Adaptive verification reduces total decision time relative to static verification.

Reliance measures are reported as secondary outcomes, distinguishing true reliance on the LLM from agreement with an answer the participant already held. Neither task type nor task difficulty is an input to the adaptive policy; both are analysed only as exploratory moderators of policy effectiveness. A policy premise check examines whether initial confidence is associated with initial answer cor-

rectness, and a non-inferiority test establishes whether adaptive verification preserves decision quality relative to static verification.

1.6 Approach

The thesis uses a within-subject experiment in which each participant completes eighteen introductory Python tasks under three escalation policies: no verification, static verification, and confidence-gated adaptive verification. Condition order is counterbalanced. Recommendations are produced offline by a frozen LLM snapshot and pre-validated so that each condition contains a balanced mix of correct and incorrect advice; verification artefacts are derived from the task specification and executed independently. The two primary outcomes are final-answer accuracy and total decision time, with reliance measures reported as secondary outcomes. Trial-level mixed-effects models account for the nesting of trials within participants and tasks, and accuracy is assessed by a non-inferiority test while effort is assessed by a superiority test. The instrumented prototype records evidence availability and examination so that the policy can be reproduced and audited.

1.7 Contributions and Scope

The novelty is *not* the observation that confidence relates to reliance on AI—miscalibration is well documented [8], [15] and taken here as a premise—but the combination of elements usually studied in isolation: (i) confidence as a per-trial *trigger* deciding *when* support is shown, rather than an outcome to be calibrated or an AI-side uncertainty signal; (ii) *independent, execution-based* evidence (tests, traces, counterexamples) rather than additional AI-generated expla-

nations; (iii) a programming setting in which recommendations are pre-validated and objective evidence exists; and (iv) an explicit effort–quality trade-off evaluation, treating verification effort as a first-class outcome—in contrast to prior work that measures accuracy, reliance, or system cost [11], [12].

The thesis is expected to contribute: (1) trial-level empirical evidence, analysed with mixed-effects models, on whether a simple confidence-based policy reduces verification effort while preserving decision quality (RQ1–RQ2); (2) an explicit escalation rule that states who recommends, who provides evidence, who decides, and what is recorded, and that treats the three conditions as three escalation policies within one specification [14]; (3) a validated benchmark of introductory Python tasks with fixed, pre-validated LLM recommendations (balanced correct/incorrect), task-type and difficulty labels, and verification data; (4) a reproducible prototype of the three conditions with complete logging, including evidence examination; and (5) an analysis pipeline for the primary outcome and effort measures.

The contribution is deliberately not a new LLM, a new algorithm, or a tutoring system: the confidence gate is intentionally simple, and its value lies in the controlled empirical comparison against always-available and no-verification baselines in a reproducible human–AI interaction experiment.

Taken together, these contributions show how a minimal user-side signal can govern when independent evidence is worth its cost. If the policy is supported, it offers programming environments a model-agnostic way to reduce interruption without weakening decisions; if it is not, the boundary conditions delimit when confidence gating is unsafe. Either outcome informs the design of verification support for novice programmers.

1.8 Structure of the Thesis

The remainder of the thesis is organised as follows. Chapter 2 reviews the literature on trust and appropriate reliance, explanation-based, cognitive-forcing, and selective interventions, verification and its cost, confidence as a signal for adaptive support, the coordination and governance of LLM-based systems, and LLM assistance in programming education, and derives the research gap addressed here. Chapter 3 presents the methodology: the benchmark and task domain, the recommendation specification, the verification artefacts, the three escalation policies, and the outcome measures. Chapter 4 describes the implementation of the instrumented prototype and its logging. Chapter 5 reports the evaluation, the statistical analysis, the discussion of results, threats to validity, and implications. Chapter 6 concludes and outlines future work.

Chapter 2

Literature Review

2.1 Introduction and Review Scope

Research on human–AI decision-making spans human factors, explainable AI, human–computer interaction, and computing education. Reviewing these literatures separately would obscure the tension motivating this thesis: an intervention that improves trust in one setting can degrade the joint decision in another, so the relevant question is not whether users trust a system but whether they rely on it appropriately. The review is organised around four strands—trust and appropriate reliance; explanation-based, cognitive-forcing, and adaptive interventions; verification, confidence, and coordination; and LLM assistance in programming education—and for each it classifies the approaches, evaluates the evidence, and states the implications for this study. Sources came from targeted, iterative searches of the major computing and HCI databases, prioritising peer-reviewed work from 2019–2027 and including older foundational work where it defines a construct used here [4], [15]. The review is argument-driven rather than systematic, and its final set is restricted to studies that report an empirical human–AI

interaction result, define a construct, metric, or design framework used in the thesis, or describe a comparable educational setting.

2.2 Trust, Reliance, and Calibration

Trust in automation is classically defined as the attitude that an agent will help an individual achieve a goal under uncertainty and vulnerability [4]. Lee and See's contribution is not only the definition but the design goal that follows from it: the objective is not maximal trust but *appropriate* reliance, calibrated to the actual capability of the automated agent. Appropriate reliance concerns the extent to which a decision maker relies on correct advice and refrains from relying on incorrect advice [5]. Its two failure modes are defined relative to the initial answer. *Over-reliance* occurs when a decision maker who was initially correct follows incorrect advice; *under-reliance* occurs when a decision maker who was initially incorrect rejects correct advice [5], [8]. This conditioning is essential: without it, accepting wrong advice and rejecting correct advice cannot be distinguished from cases in which the initial answer was already wrong or already correct.

The distinction matters because interventions that reduce over-reliance may inadvertently increase under-reliance, potentially without improving overall reliance appropriateness [8]. The construct is not merely terminological: how it is operationalized determines which errors a study can detect. Schemmer et al. observe that appropriate reliance still lacks a common definition and an operational measurement concept; their two-dimensional formulation, adopted here, separates reliance on correct advice from reliance on incorrect advice and reinforces the need to state explicitly which errors a policy is meant to reduce.

Empirical work in AI-assisted decision-making has shown that users fre-

quently fail to calibrate reliance to recommendation quality. In a sequence of pre-registered experiments with functionally identical models that varied only in apparent interpretability, participants did not follow predictions more closely when the model seemed clearer, and in some conditions were *less* able to detect the model's mistakes, apparently because of information overload [6]. Chen et al. [16] traced this result to the interaction between explanations and users' own intuition about the outcome. Feature-based explanations were visually dominant: participants scanned highlighted spans rather than reading, more often reported weak intuition about the outcome, and deferred to the AI when they could not reason about the highlighted features, so decision outcomes did not improve and over-reliance increased. Example-based explanations left the instance and the user's prior judgment in the foreground, so strong intuition could be used to override the AI and the explanation could be ignored, supporting complementary performance. Bo et al. [8] benchmarked three interventions with LLM advice in a randomised experiment with 400 participants: a static reliance disclaimer appended to the advice, token-level uncertainty highlighting, and an implicit answer that withheld the recommendation and required participants to derive it. All three reduced over-reliance, but only the disclaimer significantly improved appropriate reliance, and only on the LSAT task; in numerical estimation no intervention did, and participants became more confident after incorrect reliance decisions in some contexts. These studies establish that over- and under-reliance are persistent, not incidental, and that adding information does not by itself produce calibration.

A second body of work concerns the relation between confidence and correctness. Kruger and Dunning [15] reported that less competent individuals tend to overestimate their own performance relative to more competent peers, a pattern usually described as a metacognitive deficit. The finding is often invoked

as evidence that self-assessment is unreliable, but the underlying phenomenon is broader and more differentiated: Moore and Healy [17] distinguish overestimation of one's absolute performance, overplacement relative to others, and overprecision in the certainty of one's beliefs, and show that these components do not always move together. For the present thesis, the relevant implication is narrower than the popular Dunning–Kruger narrative: confidence is informative about correctness on average, but it is not a direct readout of correctness, and its diagnostic value must be checked empirically in the task domain rather than assumed.

These strands jointly define the outcome vocabulary used in this thesis. At the trial level, *appropriate reliance* occurs when the participant was initially incorrect, received correct advice, and adopted it, or was initially correct, received incorrect advice, and rejected it. *Over-reliance* is the proportion of trials on which the participant was initially correct, received incorrect advice, and adopted it; *under-reliance* is the proportion of trials on which the participant was initially incorrect, received correct advice, and rejected it. Conditioning on the initial answer also separates reliance from mere agreement: when the participant already holds the recommended answer, agreement with the LLM is not evidence of reliance on it. This is why the thesis distinguishes *agreement* ($F_i = L_i$, the final answer matches the recommendation) from *LLM influence* ($A_i \neq L_i$ and $F_i = L_i$, the participant moved toward the recommendation from a different prior answer) and treats stable prior agreement as a separate case. The distinction is developed further in the chapter's discussion of evaluation metrics.

2.3 Interventions That Support Reliance

Research on supporting reliance can be classified by the mechanism it targets: the information shown to the user, the interaction that forces deliberation, the expectations with which the user enters the task, or the policy that decides when support is provided. These families are not mutually exclusive, but the distinction is useful because they rest on different assumptions about why reliance fails and impose different costs on the user.

2.3.1 Explanations and Explainable AI

Explanations are the most extensively studied intervention. The rationale is that revealing why a model produced a recommendation should help users detect errors and calibrate their trust. The empirical record is more conditional. A meta-analysis of nine articles covering 44 studies and a range of explanation types—feature importance, example-based explanations, and confidence scores—found that XAI-assisted decisions outperformed unaided human decisions by 0.59 standard deviations (95% CI 0.39–0.79). In the narrower subset of studies that compared explanations with an AI recommendation alone, however, the difference was only 0.07 (95% CI -0.15–0.30): the pooled estimate concealed substantial heterogeneity, with a prediction interval ranging from -0.59 to 0.74, so explanations helped in some settings and hurt in others [10]. The gain over unaided humans was concentrated in text tasks (0.72) and absent for tabular ones (0.01). A qualitative reading of the same studies concluded that explanations mainly raised acceptance of AI advice, with only one article reporting evidence of appropriate reliance. This distinction matters: explanations may make an AI system more acceptable without making its advice easier to evaluate.

Vasconcelos et al. [7] offer a cost–benefit account that reconciles some of the mixed results. Across five studies with 731 participants, they manipulated task difficulty, explanation difficulty, and monetary incentives, and found that explanations reduce over-reliance when they sufficiently lower the cost of verifying the prediction. When checking an explanation is as costly as solving the task, users rationally fall back on the recommendation. The benefit of an explanation therefore lies in the verification effort it saves, not in its presence.

Explanation format also matters. Chen et al. [16] compared feature-based and example-based explanations in a think-aloud study and found that feature-based explanations increased over-reliance, whereas example-based explanations supported complementary human–AI performance. Their account centres on whether an explanation displaces or scaffolds the user's existing intuition about the outcome: feature-based displays disrupted it, whereas example-based displays preserved it and supplied independent signals of AI unreliability. Zhang et al. [11] sharpened this point by manipulating decision difficulty directly. Feature-based explanations reduced over-reliance for easy decisions but not for hard ones, and for the hardest decisions they could increase it. Their title states the underlying constraint: users can only verify when they already know the answer. Bansal et al. [18] reached a compatible conclusion from the perspective of team performance: explanations increased the chance that participants accepted the AI's recommendation regardless of whether it was correct, and did not increase complementary team performance. Wang et al. [19] make the broader design argument: an explanation should be chosen for the reasoning goal it serves and the cognitive bias it is meant to counter, rather than from a fixed list of explanation types or from whatever XAI technique happens to be available.

Taken together, these studies show that explanations are neither necessary

nor sufficient for appropriate reliance. They can help when they reduce the cost of checking a claim the user is able to check, and they can hurt when they persuade through fluency rather than evidence. This is one reason the present thesis separates explanation from verification: an explanation is generated by or about the model, whereas verification evidence is obtained from the task itself, independently of the model.

2.3.2 Cognitive Forcing and Expectation Setting

A second family changes the interaction rather than the content. Cognitive forcing functions change when and how a recommendation becomes available, so that accepting it requires an act of engagement rather than a passive read. The three designs tested by Buçinca et al. [9] illustrate the range: participants had to request the AI's suggestion explicitly, to commit to an initial answer before seeing it and then revise, or to wait 30 seconds while the system appeared to process the image. Compared with simple explainable-AI interfaces and a no-AI baseline in an experiment with 199 participants, these interventions significantly reduced over-reliance but did not eliminate it, and the designs that reduced over-reliance most received the least favourable subjective ratings. The benefit was also unevenly distributed: participants higher in Need for Cognition benefited more, which the authors report as an intervention-generated inequality. Cognitive forcing therefore trades effectiveness against acceptability and may widen differences between users.

Expectation setting acts earlier, before the interaction begins. Kocielnik et al. [20] studied a scheduling assistant whose AI component had the same 50% accuracy in two conditions that differed only in whether the design em-

phasised avoiding false positives or false negatives. These framings produced markedly different subjective perceptions of accuracy and acceptance, and explicit expectation-adjustment techniques increased acceptance of the imperfect system. This work is relevant because it shows that users' evaluation of an imperfect system is shaped by framing and prior expectations, not only by its accuracy. Its outcome, however, is acceptance rather than decision accuracy: baseline and expectation-setting conditions did not differ significantly in objective task accuracy (0.91 vs. 0.88), so preparing users for imperfection can make a system more acceptable without making their decisions better.

Both approaches change behaviour, but neither supplies task-grounded evidence that the user can inspect. Cognitive forcing can interrupt automatic acceptance, and expectation setting can calibrate the user's prior, yet both leave the burden of verification on the user and can reduce reliance overall rather than direct it toward correct recommendations.

2.3.3 Adaptive and Selective Interventions

A third family makes support conditional. The condition may be a property of the model, of the task, or of the user. Fanconi and van der Schaar [12] propose a cascaded LLM framework in which a base model answers first, a deferral policy sends uncertain cases to a larger model, and an abstention policy sends remaining uncertain cases to a human expert; the thresholds are updated online from human feedback. The framework balances accuracy, computational cost, and human effort, and it is evaluated on question-answering benchmarks rather than in a controlled human-subject study. Its deferral signal is the model's calibrated confidence, while abstention to the human is triggered by posterior uncertainty; and

the human receives a case to decide, not a body of evidence to evaluate.

The work closest to this thesis on the user side is that of Srinivasan and Thomason [13]. They argue that extreme trust produces under-reliance when trust is low and over-reliance when it is high, and adapt the assistant's behaviour accordingly: supporting explanations are shown when trust is low, counter-explanations when trust is high, and forced pauses are inserted to promote deliberation. In two decision-making scenarios, including medical diagnosis, supporting and counter-explanations reduced inappropriate reliance by up to 38% and improved decision accuracy by up to 20%; forced pauses separately reduced over-reliance by promoting deliberation. The signal is the user's trust state, and the gated support is still AI-generated explanation or interaction pacing, not independent verification of the recommendation.

The intervention studies reviewed above are mostly static: the same support is provided on every trial. Bo et al. [8] evaluate three such interventions with LLM advice and find that they reduce over-reliance but generally fail to improve appropriate reliance, because they reduce reliance overall rather than redirect it. The authors discuss confidence calibration as a design implication but do not use confidence as a control signal. This motivates the present work, which combines a user-side, per-trial confidence signal with independent execution-based evidence.

2.3.4 Comparative Evaluation

Table 2.1 compares the closest lines of work along four dimensions: the signal that drives support, what is gated, what the user sees, and the primary outcome. The comparison makes the gap concrete. Model-side gating uses confidence to decide whether to escalate to a more capable model or to a human [12];

TABLE 2.1
Positioning relative to the closest lines of work.

Work	Signal	What is gated	What the user sees	Primary outcome
Bo et al. [8]	None (fixed)	Nothing (uniform)	Disclaimer, uncertainty highlighting, or implicit answer	Appropriate reliance
Srinivasan and Thomason [13]	User trust state	Explanation type; forced pause	Supporting or counter-explanations	Inappropriate reliance
Zhang et al. [11]	None (difficulty as moderator)	Nothing (explanation vs. no explanation)	Feature attributions	Over-reliance
Fanconi and van der Schaar [12]	Calibrated confidence; posterior uncertainty	Large-model escalation; human review	Case only (no evidence)	Cost and accuracy
This thesis	Prior-answer confidence	Independent execution evidence	Tests, traces, counterexamples	Accuracy and effort

trust-adaptive gating uses a user state to choose an explanation or a pause [13]; the intervention studies apply their support uniformly [8]; and Zhang et al. vary decision difficulty but do not gate on it, finding that feature explanations reduce over-reliance only for easy decisions [11]. Within the scope of the search described in Section 2.1, no study has been found that gates *independent, execution-based* evidence on the decider's own prior confidence, and none has treated interaction effort as a primary outcome alongside decision quality.

The comparison also exposes a methodological difference. The studies above measure decision quality, reliance, or computational cost; CascadedLM measures human involvement as a cost to be minimised, but not the interaction effort of the person who receives the case. The present thesis treats verification effort as a first-class outcome because a policy that preserves accuracy by making everyone examine evidence on every trial has not solved the problem it set out to solve.

2.4 Independent Verification and Its Cost

Verification here means checking a claim against evidence that is independent of the source that produced it. In programming, the evidence can be a test suite, an execution trace, or a counterexample derived from the task specification and the reference answer. This differs from an explanation in kind, not only in presentation. An explanation is an account of why the model produced an answer; verification evidence is a check on whether the answer satisfies the task. The distinction is important because a fluent explanation can increase acceptance without increasing correctness [18], and because an explanation of a model's reasoning need not faithfully describe the computation that produced the answer.

Verification is not free. Human scrutiny of AI outputs consumes time and attention, and the cost is incurred on every trial on which evidence is shown. Vasconcelos et al. [7] formalise this as a cost–benefit choice: users weigh the cognitive effort of verifying the AI against the lower-effort strategy of relying on it, and engage with an explanation when the reduction in verification cost justifies the effort. CascadedLM [12] applies a related cost logic at the system level, routing a case to human review only after a larger model still returns a high-uncertainty answer. Zhang et al. [11] add a constraint that matters for novices in particular: verification presupposes an independent reference against which the answer can be judged. Feature-based explanations reduced overreliance only on decisions participants could already evaluate themselves, and not on difficult ones. Evidence that cannot be checked within the decision-time budget is not effective support, however objective it is.

Introductory programming is a favourable setting for independent verification. Tasks come with executable test suites, so correctness can be checked by

running tests rather than by judging surface similarity; code-generation benchmarks adopt functional correctness for exactly this reason [21]. The tasks are small enough that a trace or a failing test can plausibly be interpreted within the decision-time budget, and the evidence can be derived from the task specification rather than from the recommender. The evidence is still not exhaustive: a program can pass a finite test suite and remain incorrect, and a trace shows what happened on one input rather than on all inputs. Execution-based evidence therefore supports judgment rather than replacing it, which is consistent with leaving decision authority with the participant.

The design problem is consequently not how to construct verification evidence but *when* to present it. Showing evidence on every trial pays the cost even when the recommendation is already accepted correctly; withholding it entirely leaves the participant with fluency as the only cue. The next section considers whether the participant's own confidence can serve as the signal that decides when the cost is worth paying.

2.5 Confidence as a Signal for Adaptive Support

Lee and See [4, p. 70] observe that static approaches used to identify miscalibration of self-confidence in decision making "address only cumulative experience, rather than the evolving experience and continuous recalibration that are critical for appropriate reliance on automation." The implication drawn here is that calibration should not be treated as a stable property to be estimated once and then applied as a fixed weight; what matters is the relation between confidence and correctness as it evolves across tasks, trials, and moments of an interaction.

This distinction matters for adaptive verification. A single calibration score,

or a disposition measured before the task, cannot distinguish a trial on which a student is correctly confident from one on which the same student is confidently wrong. A per-trial confidence signal can, in principle, make that distinction at the point where support is being considered. Throughout this thesis, confidence refers to the participant's metacognitive judgement about the correctness of their own prior answer, elicited before the recommendation is shown; it is distinct both from trust in the assistant [4] and from the model's own uncertainty. The thesis therefore adopts the weaker, testable implication of Lee and See's argument: confidence is elicited on every trial and used as a routing signal, while its relationship to initial-answer correctness is checked empirically rather than assumed. The stronger claim—that the decision threshold itself should be continuously recalibrated—is outside the scope of the present design.

Empirically, the premise is plausible but not guaranteed. Bo et al. [8] found that interventions reduced over-reliance but generally failed to improve appropriate reliance, and that confidence after incorrect reliance decisions was poorly calibrated in some contexts. The authors discuss confidence calibration as a design implication but do not use confidence as a control signal. Related work adapts support to the user's trust state [13], adjusting the interaction to the user's evolving disposition toward the system.

In operational terms, confidence is an *input signal*—a lightweight trigger deciding *when* independent support is shown—rather than a fully calibrated internal state. This is a deliberately modest operationalisation of the dynamic view: confidence is measured repeatedly and used at the trial level, but the threshold is fixed rather than learned online. The documented relationship between confidence and correctness is treated as a premise: if confidence carries information about correctness, it can also inform *when* verification is worth its interaction cost. The

interaction-cost dimension is largely absent from calibration-oriented work and is a primary outcome here.

2.6 Coordination and Governance of LLM-Based Systems

Because a language model can produce a fluent answer without guaranteeing that it is correct, in scope, or compatible with other decisions, plausible but unsupported output must be managed by the surrounding system rather than trusted on fluency alone [14]. Recent work on agentic software engineering argues that increasing autonomy raises, rather than lowers, the need for coordination: claims may need to be checked against artefacts, actions against permissions, and decisions against explicitly assigned authority, and shared artefacts, logs, tests, and approval points make decisions traceable [14].

This perspective motivates the explicit escalation rule adopted in this thesis (Section 1.4): coordination concerns who decides, with what evidence, and under whose authority, and here that allocation is made explicit in independent, execution-based evidence that replaces reliance on conversational output alone. It also clarifies why verification is not merely a UX feature: it is the mechanism by which the system, rather than the fluency of the model, governs when a recommendation is trusted.

2.7 LLMs in Programming Education

LLMs are increasingly used for code generation, debugging, and explanation in computing education [1]–[3]. They can accelerate routine work and provide on-demand feedback, but they can also be plausible yet incorrect, so learners must evaluate recommendations critically. Prior work establishes that intuition, explanation design, and decision difficulty shape reliance [7], [11], [16]. In educational settings, the stakes of miscalibration are high: accepting an incorrect recommendation can entrench a misconception, while rejecting a correct one wastes an opportunity to learn.

2.8 Synthesis and Research Gap

The reviewed literature converges on three points. First, appropriate reliance—not trust maximisation—is the correct objective, and over- and under-reliance are persistent failure modes [4], [6], [8]. Second, explanations and cognitive forcing help only conditionally, and information alone does not guarantee better monitoring [7], [9], [10]. Third, human scrutiny is costly and should be invoked selectively [12], and explanation-based support helps mainly on decisions users can already evaluate themselves [11].

What remains unknown is whether a learner's *initial confidence* can serve as a lightweight trigger for selectively displaying *independent verification evidence* after an LLM recommendation in introductory programming tasks—reducing verification effort relative to always-available support while preserving final-answer accuracy and appropriate reliance. This is the gap the thesis addresses.

Since LLM-based assistants have entered programming education, the cost

of human scrutiny has become a primary design constraint, and the reviewed literature points to a controlled comparison of escalation policies rather than to a new model or algorithm. Accordingly, the policy is tested in a within-subject experiment in which all participants experience all three escalation policies, so that trial-level comparisons are not confounded by individual differences. Trial-level outcomes are analysed with mixed-effects models because trials are nested within participants and tasks. Decision quality is assessed with a non-inferiority test rather than a superiority test, because the cost literature reviewed here indicates that the goal is to preserve quality while reducing verification effort. Recommendations are frozen and pre-validated so that the verification policy, not model behaviour, is the manipulated factor. Concretely, the design treats confidence as an escalation signal within a single explicit escalation rule, measures verification effort as a first-class outcome, and evaluates decision quality against explicit non-inferiority criteria, including the boundary case of participants who are confident but mistaken.

References

[1] Q. Ma, H. Shen, K. Koedinger, and S. T. Wu, "How to teach programming in the ai era? using llms as a teachable agent for debugging," in *Artificial Intelligence in Education: 25th International Conference, AIED 2024, Recife, Brazil, July 8–12, 2024, Proceedings, Part I*, Berlin, Heidelberg: Springer-Verlag, 2024, pp. 265–279, ISBN: 978-3-031-64301-9. DOI: 10.1007/978-3-031-64302-6_19.

[2] D. Cambaz and X. Zhang, "Use of ai-driven code generation models in teaching and learning programming: A systematic literature review," in *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*, New York, NY, USA: Association for Computing Machinery, 2024, pp. 172–178. DOI: 10.1145/3626252.3630958.

[3] P. Denny, J. Prather, B. A. Becker, *et al.*, "Computing education in the era of generative ai," *Commun. ACM*, vol. 67, no. 2, pp. 56–67, Jan. 2024, ISSN: 0001-0782. DOI: 10.1145/3624720.

[4] J. D. Lee and K. A. See, "Trust in automation: Designing for appropriate reliance," *Human Factors*, vol. 46, no. 1, pp. 50–80, 2004. DOI: 10.1518/hfes.46.1.50_30392.

[5] M. Schemmer, N. Kuehl, C. Benz, A. Bartos, and G. Satzger, "Appropriate reliance on ai advice: Conceptualization and the effect of explanations," in *Proceedings of the 28th International Conference on Intelligent User Interfaces*, New York, NY, USA: Association for Computing Machinery, 2023, pp. 410–422, ISBN: 9798400701061. DOI: 10.1145/3581641.3584066.

[6] F. Poursabzi-Sangdeh, D. G. Goldstein, J. M. Hofman, J. Wortman Vaughan, and H. Wallach, "Manipulating and measuring model interpretability," in *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, ser. CHI '21, Yokohama, Japan: Association for Computing Machinery, 2021, ISBN: 9781450380966. DOI: 10.1145/3411764.3445315.

[7] H. Vasconcelos, M. Jörke, M. Grunde-McLaughlin, T. Gerstenberg, M. Bernstein, and R. Krishna, "Explanations can reduce overreliance on ai systems during decision-making," *Proceedings of the ACM on Human-Computer Interaction*, vol. 7, pp. 1–38, 2022. DOI: 10.1145/3579605.

[8] J. Y. Bo, S. Wan, and A. Anderson, "To rely or not to rely? evaluating interventions for appropriate reliance on large language models," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, ser. CHI '25, New York, NY, USA: Association for Computing Machinery, 2025. DOI: 10.1145/3706598.3714097.

[9] Z. Buçinca, M. B. Malaya, and K. Z. Gajos, "To trust or to think: Cognitive forcing functions can reduce overreliance on ai in ai-assisted decision-making," *Proc. ACM Hum.-Comput. Interact.*, vol. 5, no. CSCW1, Apr. 2021. DOI: 10.1145/3449287.

[10] M. Schemmer, P. Hemmer, M. Nitsche, N. Kühl, and M. Vössing, "A meta-analysis of the utility of explainable artificial intelligence in human-ai decision-making," in *Proceedings of the 2022 AAAI/ACM Conference on AI, Ethics, and Society*, ser. AIES '22, Oxford, United Kingdom: Association for Computing Machinery, 2022, pp. 617–626, ISBN: 9781450392471. DOI: 10.1145/3514094.3534128.

[11] Z. T. Zhang, F. Buchner, Y. Liu, and A. Butz, "You can only verify when you know the answer: Feature-based explanations reduce overreliance on ai for easy decisions, but not for hard ones," in *Proceedings of Mensch Und Computer 2024*, ser. MuC '24, Karlsruhe, Germany: Association for Computing Machinery, 2024, pp. 156–170. DOI: 10.1145/3670653.3670660.

[12] C. Fanconi and M. van der Schaar, "Cascaded language models for cost-effective human-AI decision-making," in *Advances in Neural Information Processing Systems*, vol. 38, Curran Associates, Inc., 2025. DOI: 10.52202/085713-0388. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2025/hash/10e0c427408ccc6e07Abstract-Conference.html.

[13] T. Srinivasan and J. Thomason, "Adjust for trust: Mitigating trust-induced inappropriate reliance on ai assistance," in *Proceedings of the 31st International Conference on Intelligent User Interfaces*, New York, NY, USA: Association for Computing Machinery, 2026, pp. 1883–1900. DOI: 10.1145/3742413.3789136.

[14] P. Ciancarini, "Who is in charge? coordination and governance in agentic software engineering," *Journal of Logical and Algebraic Methods in Pro-*

gramming, vol. 152, p. 101 192, 2027, ISSN: 2352-2208. DOI: 10.1016/j.jlamp.2026.101192.

[15] J. Kruger and D. Dunning, "Unskilled and unaware of it: How difficulties in recognizing one's own incompetence lead to inflated self-assessments," *Journal of Personality and Social Psychology*, vol. 77, no. 6, pp. 1121–1134, 1999. DOI: 10.1037/0022-3514.77.6.1121.

[16] V. Chen, Q. V. Liao, J. Wortman Vaughan, and G. Bansal, "Understanding the role of human intuition on reliance in human-ai decision-making with explanations," *Proc. ACM Hum.-Comput. Interact.*, vol. 7, no. CSCW2, Oct. 2023. DOI: 10.1145/3610219.

[17] D. A. Moore and P. J. Healy, "The trouble with overconfidence.," *Psychological review*, vol. 115 2, pp. 502–17, 2008. [Online]. Available: https://api.semanticscholar.org/CorpusID:14108721.

[18] G. Bansal, T. Wu, J. Zhou, *et al.*, "Does the whole exceed its parts? the effect of ai explanations on complementary team performance," in *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, New York, NY, USA: Association for Computing Machinery, 2021, ISBN: 9781450380966. DOI: 10.1145/3411764.3445717.

[19] D. Wang, Q. Yang, A. Abdul, and B. Y. Lim, "Designing theory-driven user-centric explainable ai," in *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, New York, NY, USA: Association for Computing Machinery, 2019, pp. 1–15, ISBN: 9781450359702. DOI: 10.1145/3290605.3300831.

[20] R. Kocielnik, S. Amershi, and P. N. Bennett, "Will you accept an imperfect ai? exploring designs for adjusting end-user expectations of ai systems," in *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, New York, NY, USA: Association for Computing Machinery, 2019, pp. 1–14, ISBN: 9781450359702. DOI: 10.1145/3290605.3300641.

[21] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation," in *Advances in Neural Information Processing Systems*, vol. 36, Curran Associates, Inc., 2023, pp. 21 558–21 572. DOI: 10.52202/075280-0943. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/43e9d647ccd3e4b7b5baab53f0Paper-Conference.pdf.